Designing a Fibonacci Accelerator

Authors:
WhisperingRock

Class
CPE490: Open-Source ASIC Design
November 3, 2026

Contents

Contents . . . 1

1 Introductions **2**

2 v0 Pre-Synthesis **3**
2.1 Procedure . . . 3
2.2 Design . . . 3
2.3 Simulation Testing . . . 9
2.3.1 Brent-Kung 'Parallel Prefix' Adder . . . 9
2.3.2 Fibonacci Accelerator . . . 10

3 v0 Post-Synthesis Analysis **11**
3.1 Static Timing Analysis (Pre-PlaceNRoute) . . . 11
3.1.1 Analysis . . . 12

4 v1 Redesign + Analysis **13**
4.1 v1 Post-Synthesis Analysis . . . 15
4.2 v1 meeting STA . . . 17

5 Tile Area **19**

6 Appendix **20**
6.1 Code . . . 20
6.1.1 RTL . . . 20
6.1.2 Simulation SVA Testing . . . 26

7 References **29**

Chapter 1

Introductions

In this lab, the task was to create a Fibonacci accelerator. Before going further in the paper, I must first describe the Fibonacci sequence and what makes a design an *accelerator*.

First, let's discuss the Fibonacci sequence using a recursive formula.

$$\boldsymbol{F_{n+2} = F_{n+1} + F_n} \text{ where } \boldsymbol{F_0 = 0} \text{ and } \boldsymbol{F_1 = 1}$$

In our discussion in class, another relationship emerged when comparing multiple terms.

Consider:

- Let $F_n = a$
- Let $F_{n+1} = b$
- $F_{n+2} = a + b$
- $F_{n+3} = a + 2b$
- $F_{n+4} = 2a + 3b$
- $F_{n+5} = 3a + 5b$

If given two terms of the sequence, we can potentially calculate the next four terms from the initial two. This unique way to describe terms has potential for a speed-up if we can utilize shifters and adders with the motivation to calculate many terms in a single cycle.

This motivation to increase throughput or performance over a CPU, through the use of configurable logic, is termed as an *accelerator*. For this lab, our accelerator will calculate more than one term of the Fibonacci sequence per clock cycle.

Lastly, another motivation of this lab is to become familiar with open-source tooling set.

Chapter 2

v0 Pre-Synthesis

2.1 Procedure

The workflow of this lab, which is largely focused on design before synthesis, will be the following

1. LINTing based off of many tools s.a. Verilator, Icarus, GL, ...
2. Simulation of test scripts on Verilator, then Icarus Verilog
3. BMC verification
4. K-Induction formal verification (optional)

2.2 Design

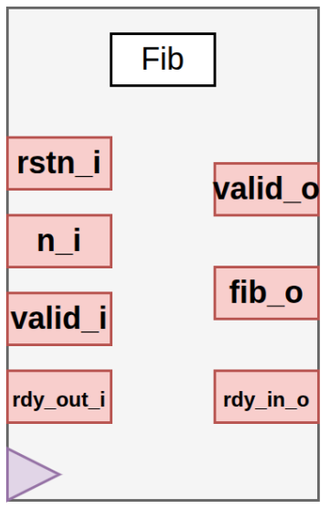


Figure 2.1: Fibonacci Top Module

The top-level module has a data and control signals:

- **clk_i**: clock (posedge is captured).
- **rstn_i**: active-low synchronous reset.
- **valid_i**: indicates when input n_i can be read.
- **n_i**: requested term of fib sequence.
- **rdy_out_i**: indicates when downstream has read this modules output (and module can begin processing a new sequence).

- **valid_o**: indicates module's output is ready to be read.
- **rdy_on_o**: indicates module is ready for a new request (n_i).
- **fib_o**: modules output (F_n).

In order to capture the functionality of these top-level signals, an FSM was used:

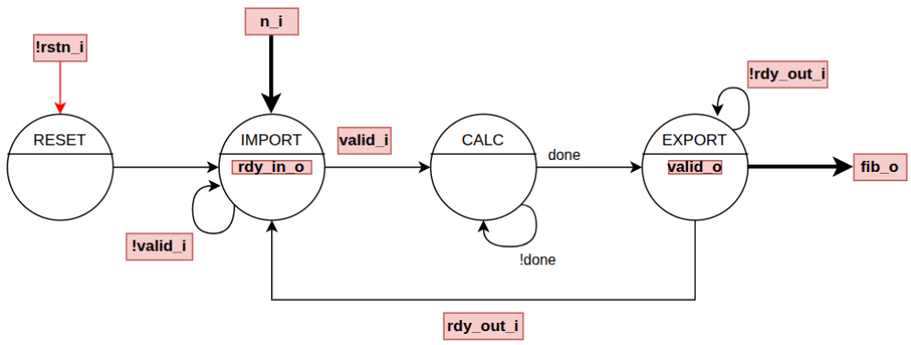


Figure 2.2: FSM of Fibonacci Accelerator

From here, lets take a closer look into the CALC stage of the FSM, where most of our accelerator logic comes into play. An attentive reader will notice that there are only two adders and a single shifter: there is a considerable amount of adders required to further calculate F_{n+4} and F_{n+5}. Since this must be implemented in ASIC where wires and hardware are not free lunches, this designer opted to take 2 cycles to calculate 4 terms of the fib sequence. This will keep the area smaller while producing two elements per cycle (a +2x increase over a CPU).

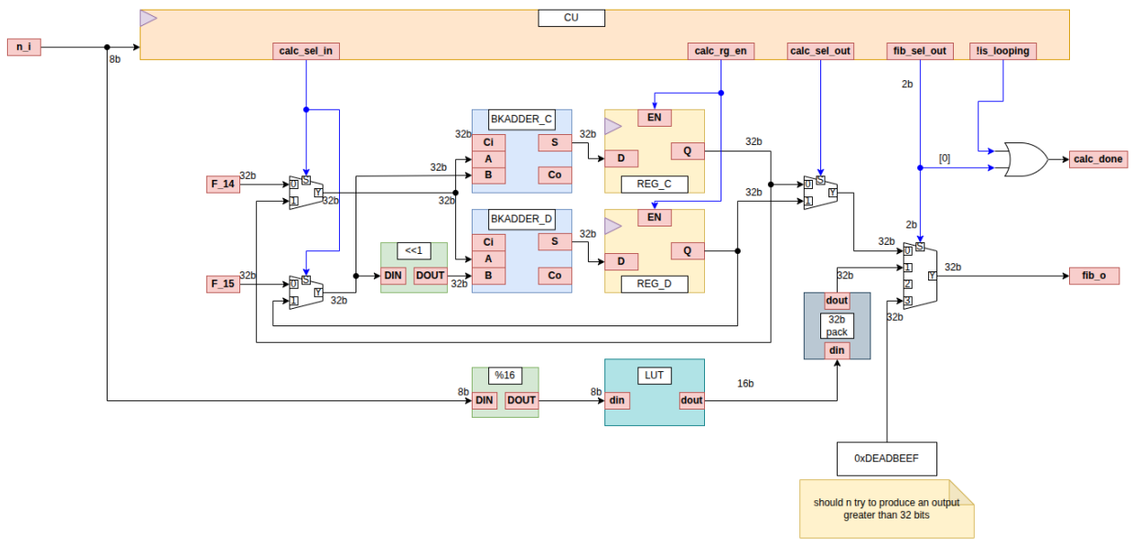


Figure 2.3: Computation Portion of Fib Accelerator

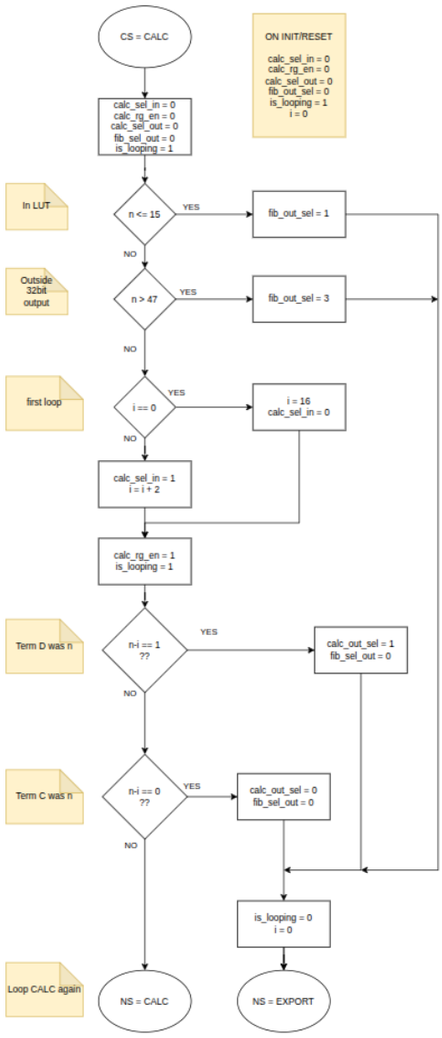
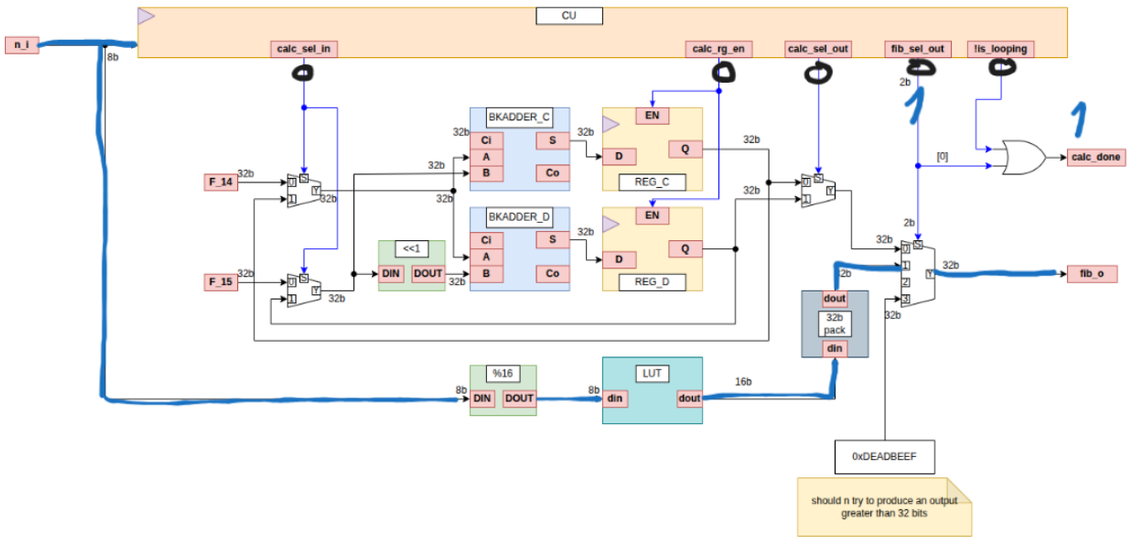


Figure 2.4: Control Flow of CALC State

To properly explain the design, let's trace through various values of n.



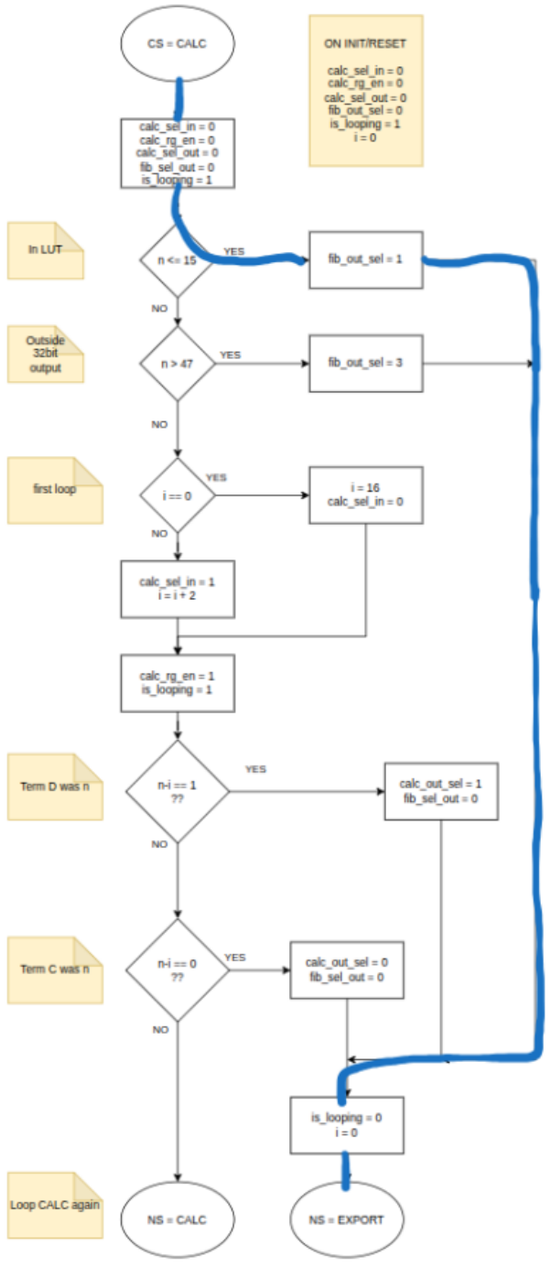
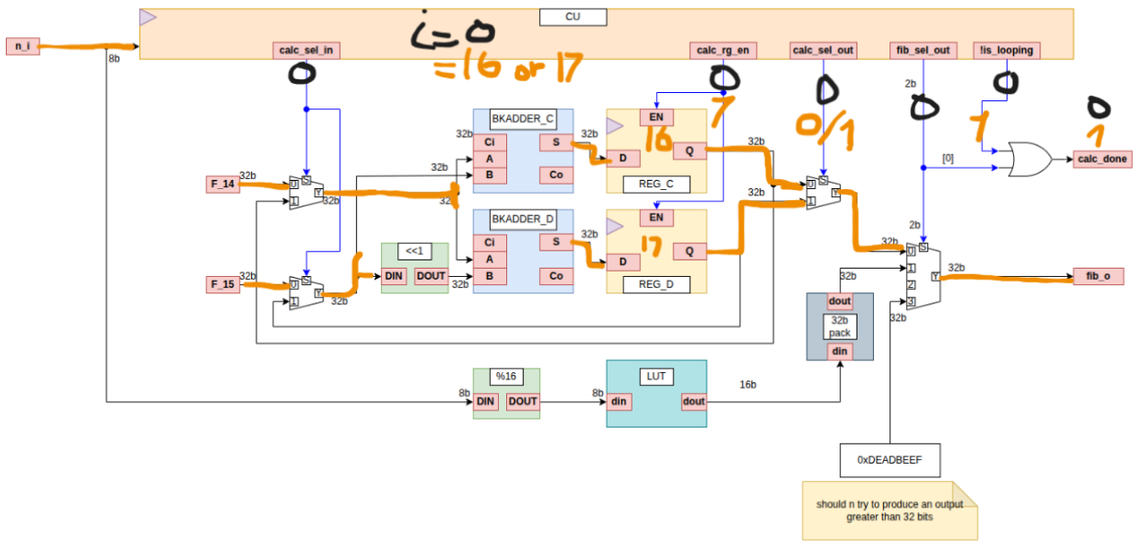


Figure 2.5: $n \leq 15$ uses the LUT



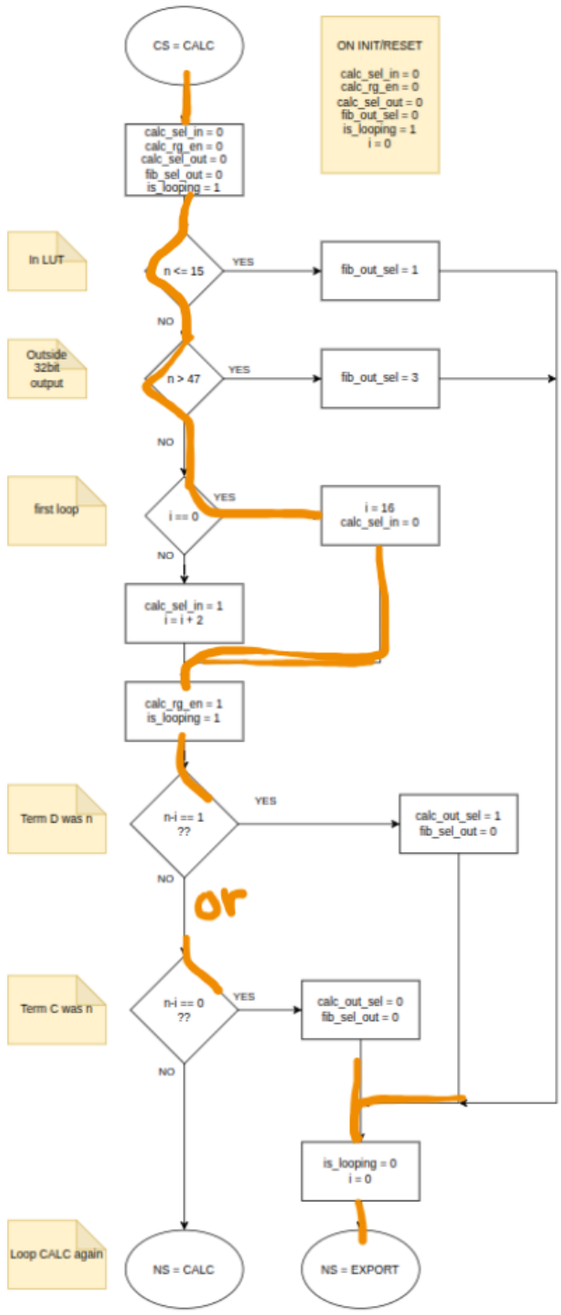


Figure 2.6: $n = 16$ or $n = 17$ calculates but doesn't loop

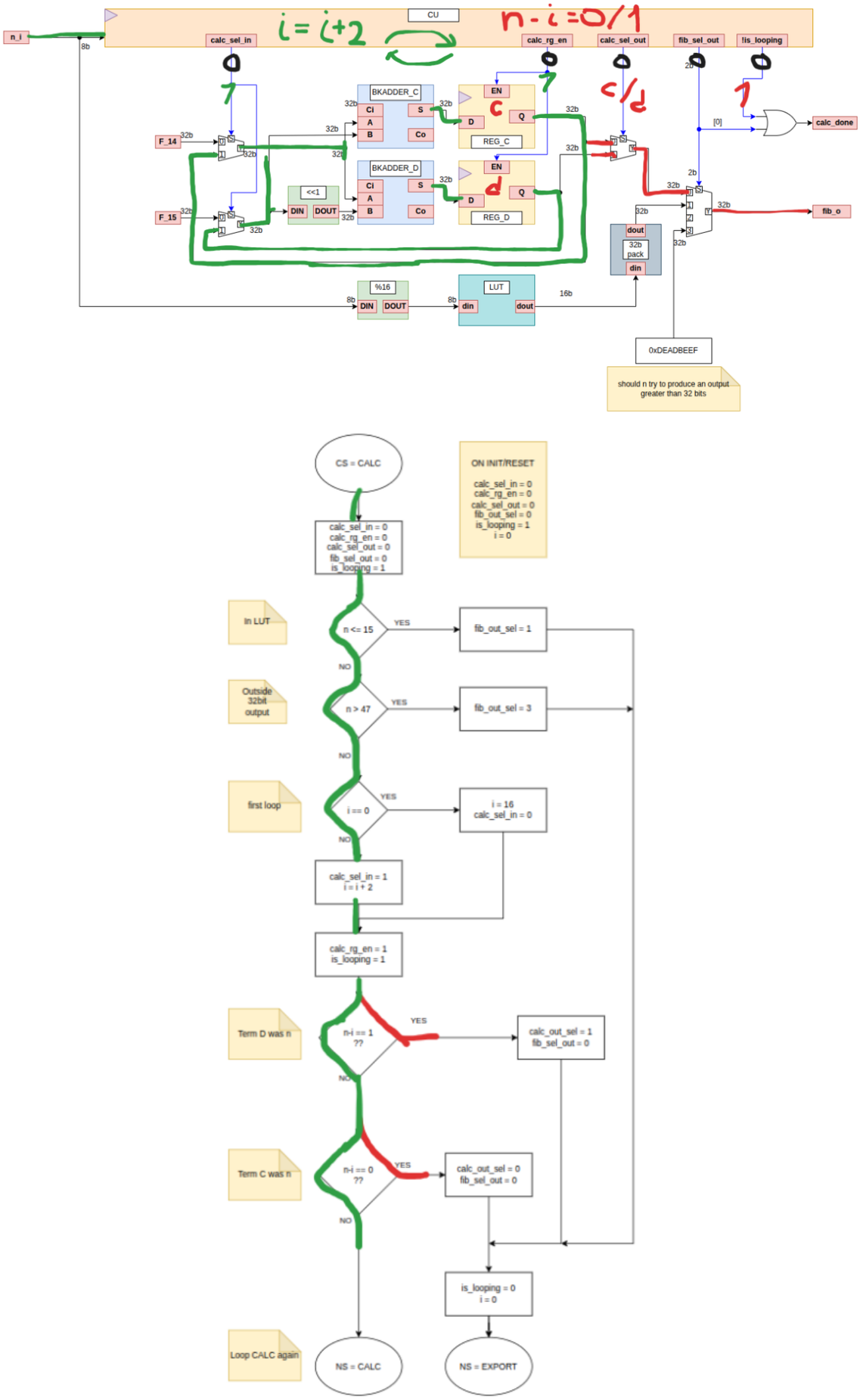


Figure 2.7: $n \geq 18$ and $n \leq 47$ calculates using loops

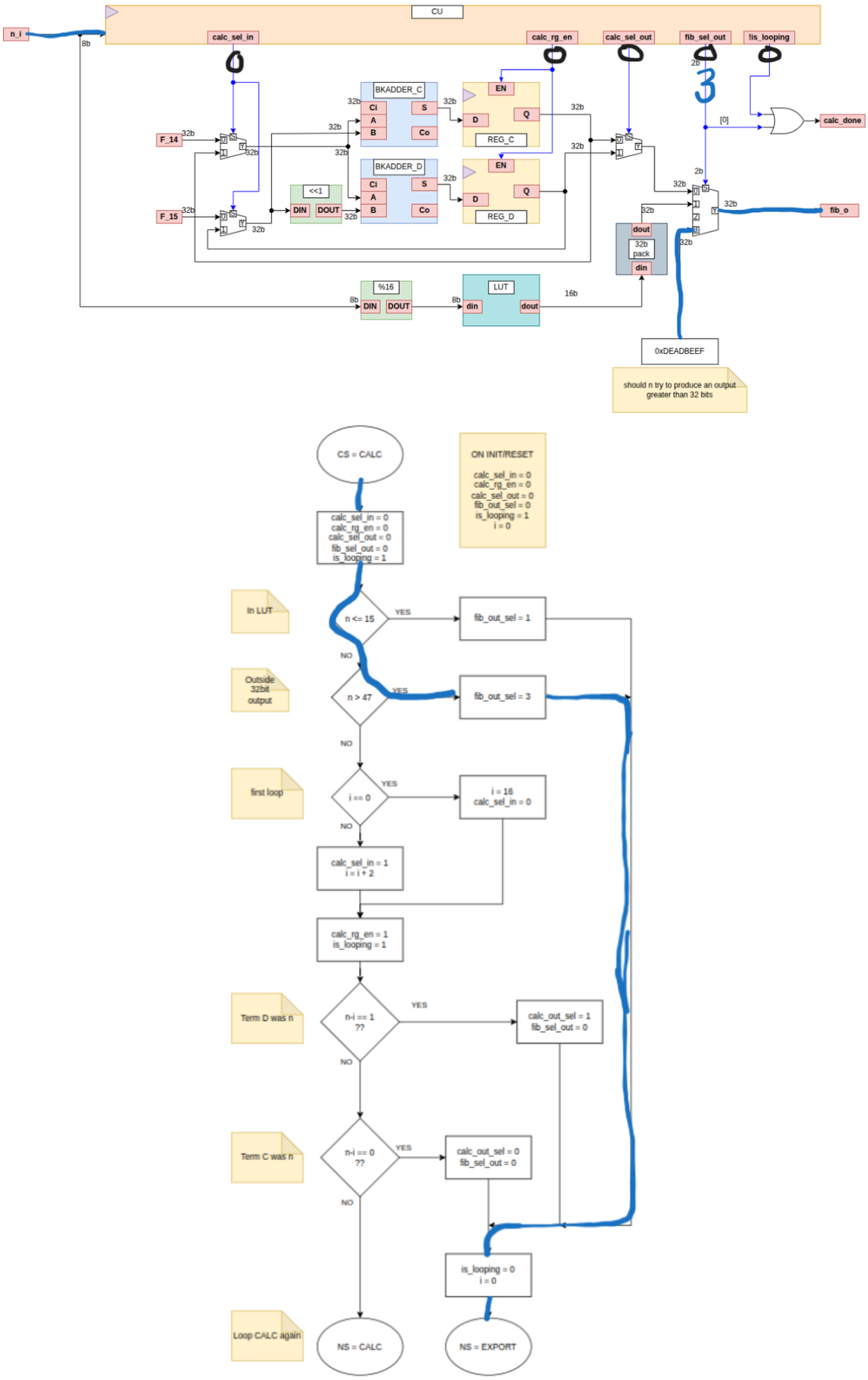


Figure 2.8: $n > 47$ defaults to avoid output overflow

2.3 Simulation Testing

2.3.1 Brent-Kung 'Parallel Prefix' Adder

The BKAdder was studied and implemented as a dynamic model, only verified using formal verification methods BMC and K-Induction. The rtl and verification can be seen in the same file 6.2.

Test Type	Result
BMC	✓
Induction	✓

Table 2.1: Verification Methods for BKAdder

2.3.2 Fibonacci Accelerator

The accelerator was initially verified using immediate and concurrent simulations on verilator (not icarus because SVA is not supported). The simulation file 6.4 tests:

- **Concurrents**
  - **reset_import**: confirms the module comes out of RESET, enters the IMPORT state, and signals the upstream that it is ready to accept an input n.
  - **import_export**: confirms the model is not holding the output steady while prompting the upstream for a new input n.
  - **hold_fib**: confirms the output `fib_o` was steady as long as our module is finished calculating and the downstream has not confirmed an output read.
- **Immediates**
  1. Test $n \in [0, 47]$ outputs the correct values sequentially.
  2. Test $n \in [48, 255]$ outputs an error code because output overflow is certain.

Test Type	Result
Verilator (SVA) Concurrents	✓
Verilator (SVA) Immediates	✓
Icarus (SVA) Immediates	✓
BMC	✗TODO
Induction	✗TODO

Table 2.2: Verification Methods for BKAdder

Chapter 3

v0 Post-Synthesis Analysis

Listing 3.1: v0 LibreLane config settings

```
---
DESIGN_NAME: fib
VERILOG_FILES: dir::rtl/*.sv
CLOCK_PERIOD: 10
CLOCK_PORT: clk_i
# Power Distribution Stuff
PDN_VOFFSET: 7
PDN_HOFFSET: 7
PDN_SKIPTRIM: true
RUN_POST_GRT_RESIZER_TIMING: true
RUN_POST_GRT_DESIGN_REPAIR: true

# Use our naming convention
SYNTH_AUTONAME: true
```

3.1 Static Timing Analysis (Pre-PlaceNRoute)

Listing 3.2: Overall Summary of Initial Post-Layout Simulation

```
Running 'Checker.SetupViolations' at 'runs/RUN_2026-10-29_15-38-45/74-checker-setupviolations'
Setup violations found in the following corners:
* max_ss_100C_1v60
* min_ss_100C_1v60
* nom_ss_100C_1v60
No setup violations found
Running 'Checker.HoldViolations' at 'runs/RUN_2026-10-29_15-38-45/75-checker-holdviolations'
No hold violations found
Running 'Checker.MaxSlewViolations' at 'runs/RUN_2026-10-29_15-38-45/76-checker-maxslewviolations'
Max Slew violations found in the following corners:
* max_ss_100C_1v60
* max_tt_025C_1v80
* min_ss_100C_1v60
* min_tt_025C_1v80
* nom_ss_100C_1v60
* nom_tt_025C_1v80
No max slew violations found
Running 'Checker.MaxCapViolations' at 'runs/RUN_2026-10-29_15-38-45/77-checker-maxcapviolations'
No max cap violations found
```

Listing 3.3: openroad-staprepnr Summary

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violatio.	Max Slew Violati.
Overall	0.0383	0.0383	0.0000	0	0	-11.7484	-11.7484	-614.05.	108	108	3	466
nom_tt_025C_1v80	0.1905	0.1905	0.0000	0	0	-1.5365	-1.5365	-32.1235	39	39	3	329
nom_ss_100C_1v60	0.6187	0.6187	0.0000	0	0	-11.7484	-11.7484	-614.05.	69	69	3	466
nom_ff_n40C_1v95	0.0383	0.0383	0.0000	0	0	3.0052	3.0052	0.0000	0	0	3	202

After performing an initial static-timing analysis, there is evidence of setup violations in the pre-place and route phase, with the biggest offending corner at `nom_ss_100C_1v60`. Since this corner is testing logic speeds at the lowest worse case, we can infer that our logic is too slow and that the longest and slowest paths cannot meet out target frequency of 10ns. Now that we've identified which corner is the biggest thorn, we can start to ask where and which path(s) are in violation.

NOTE: Lets reside in pre-pnr for now, since the likelihood our timing issues will resolve after placement is not good.

Listing 3.4: Route for Setup's Biggest Offender

```
1
2  ===========================================================================
3  report_checks -path_delay max (Setup)
4  ===========================================================================
5  ======================== nom_ss_100C_1v60 Corner ===================================
6
7  Startpoint: valid_o_sky130_fd_sc_hd__dfxtp_2_Q
8              (rising edge-triggered flip-flop clocked by clk_i)
9  Endpoint: mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q
10           (rising edge-triggered flip-flop clocked by clk_i)
11 Path Group: clk_i
12 Path Type: max
13
14 Fanout          Cap         Slew       Delay        Time   Description
15 ---------------------------------------------------------------------------------------------
16                          0.150000    0.000000    0.000000   clock clk_i (rise edge)
17                                      0.000000    0.000000   clock network delay (ideal)
18                          0.150000    0.000000    0.000000 ^ valid_o_sky130_fd_sc_hd__dfxtp_2_Q/CLK (sky130_fd_sc_hd__dfxtp_2)
19      4    0.040429     0.187477    0.807686    0.807686 v valid_o_sky130_fd_sc_hd__dfxtp_2_Q/Q (sky130_fd_sc_hd__dfxtp_2)
20                                                           valid_o (net)
21                          0.187477    0.000000    0.807686 v CS[0]_sky130_fd_sc_hd__or2_2_B/A (sky130_fd_sc_hd__or2_2)
22      4    0.011350     0.159989    0.817762    1.625448 v CS[0]_sky130_fd_sc_hd__or2_2_B/X (sky130_fd_sc_hd__or2_2)
23                                                           CS[0]_sky130_fd_sc_hd__or2_2_B_X (net)
24                          0.159989    0.000000    1.625448 v fib_sel_out[0]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a32o_2_X_A2_sky130_fd_sc_hd__or2_2_X_B_sky130_fd_sc_
25    126    0.350231    12.214338    9.953817   11.579266 ^ fib_sel_out[0]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a32o_2_X_A2_sky130_fd_sc_hd__or2_2_X_B_sky130_fd_sc_
26                                                           fib_sel_out[0]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a32o_2_X_A2_sky130_fd_sc_hd__or2_2_X_B_sky130_fd_sc_
27                         12.214338    0.000000   11.579266 ^ mux_b.DIN[46]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__nand2_2_Y_B_sky130_fd_s
28      2    0.003792     0.265631    2.905223   14.484488 ^ mux_b.DIN[46]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__nand2_2_Y_B_sky130_fd_s
29                                                           mux_b.DIN[46]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__nand2_2_Y_B_sky130_fd_s
30                          0.265631    0.000000   14.484488 ^ mux_b.DIN[45]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
31      3    0.014004     0.112177    0.565484   15.049973 v mux_b.DIN[45]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
32                                                           mux_b.DIN[45]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_A (net)
33                          0.112177    0.000000   15.049973 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
34      1    0.001248     0.147204    1.126995   16.176968 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
35                                                           mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
36                          0.147204    0.000000   16.176968 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
37      2    0.003505     0.169459    1.044334   17.221302 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
38                                                           mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
39                          0.169459    0.000000   17.221302 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
40      3    0.007130     0.257365    1.464960   18.686262 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
41                                                           mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
42                          0.257365    0.000000   18.686262 v mux_b.DIN[51]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
43      5    0.013657     0.318167    0.429750   19.116013 ^ mux_b.DIN[51]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
44                                                           mux_b.DIN[51]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
45                          0.318167    0.000000   19.116013 ^ mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
46      2    0.006094     0.215493    0.359973   19.475986 v mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
47                                                           mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
48                          0.215493    0.000000   19.475986 v mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
49      5    0.016287     0.363301    0.466146   19.942133 ^ mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
50                                                           mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
51                          0.363301    0.000000   19.942133 ^ mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2_sky130_fd_sc_hd__nand2_2_Y_B_sky130_fd_s
52      1    0.004617     0.070111    0.335543   20.277676 ^ mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2_sky130_fd_sc_hd__nand2_2_Y_B_sky130_fd_s
53                                                           mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2_sky130_fd_sc_hd__nand2_2_Y_B (net)
54                          0.070111    0.000000   20.277676 ^ mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2_sky130_fd_sc_hd__nand2_2_Y/B (sky130_fd_
55      1    0.002251     1.068041    0.090818   20.368494 v mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2_sky130_fd_sc_hd__nand2_2_Y/Y (sky130_fd_
56                                                           mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2 (net)
57                          1.068041    0.000000   20.368494 v mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X/B2 (sky130_fd_sc_hd__o221a_2)
58      1    0.001577     0.082435    0.871987   21.240480 v mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X/X (sky130_fd_sc_hd__o221a_2)
59                                                           mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D (net)
60                          0.082435    0.000000   21.240480 v mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q/D (sky130_fd_sc_hd__dfxtp_2)
61                                                  21.240480   data arrival time
62
63                          0.150000   10.000000   10.000000   clock clk_i (rise edge)
64                                      0.000000   10.000000   clock network delay (ideal)
65                                     -0.250000    9.750000   clock uncertainty
66                                      0.000000    9.750000   clock reconvergence pessimism
67                                                  9.750000 ^ mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q/CLK (sky130_fd_sc_hd__dfxtp_2)
68                                     -0.257883    9.492117   library setup time
69                                                  9.492117   data required time
70 ---------------------------------------------------------------------------------------------
71                                                  9.492117   data required time
72                                                -21.240480   data arrival time
73 ---------------------------------------------------------------------------------------------
74                                                -11.748363   slack (VIOLATED)
```

Looking at fig 3.4, there are a few items that stand out.

- L25: signal `fib_sel_out[0]` has a fanout of 126 when its rising ($\wedge$). Moreover, we occurred a massive delay and skew on this one transaction.
- L27-59: The end bits on the input mux b are DOWNSTREAM and ADDING to the delay of this path.

3.1.1 Analysis

- Q: Why is there such a large fanout on this signal? This signal is the SEL signal for the `mux_fib` in fig 2.3, meaning that it is connected to many smaller muxes to drive the 128-bit one. This, on the surface, doesn't sound too problematic if this was the only logic to be calculated. Might there be something else wrong?
- Lets scrutinize the path in fig 2.7 as I suspect its also causing unnecessary delay. First, lets outline what is actually happening here, assuming a newly calculated term is our requested one (b/c its worst case).
  1. While the CU flies through the conditional statements, the adders are beginning to calculate new terms for C and D.
  2. Register enable is high s.t. we capture these calculations.
  3. We configure **two** 32-bit muxes to deliver term D to `fib_o` as well as set `calc_done`.
  4. The adders finish and the registers capture.
  5. The output muxes receive the data on the next clock cycle.

  The initial inspiration was to store the new calculations in the registers as well as config the muxes, then by the EXPORT stage, we should hold and be ready to output the correct value. Unfortunately, this is too much logic to run through in a single stage and clock cycle. Moreover, a second 32-bit mux on the output is not needed.

Chapter 4

v1 Redesign + Analysis

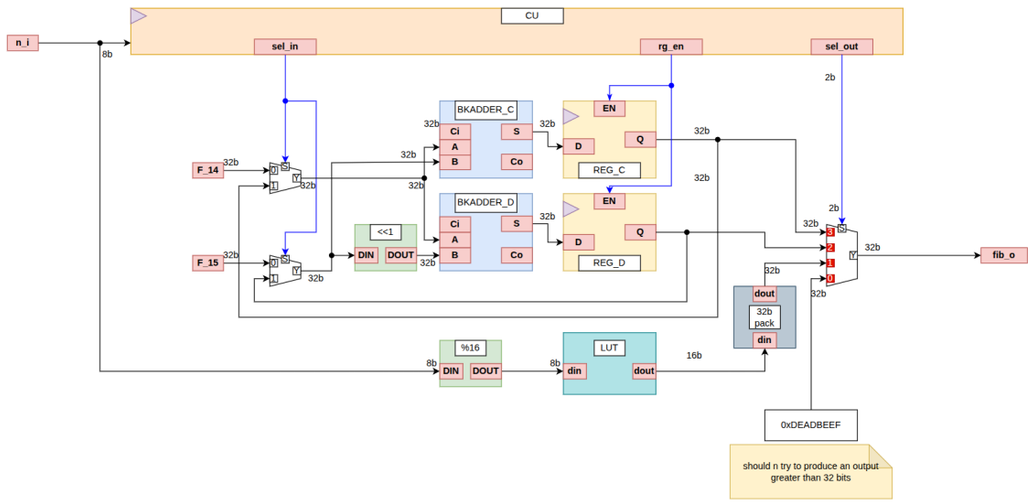


Figure 4.1: v1 Accelerator Logic

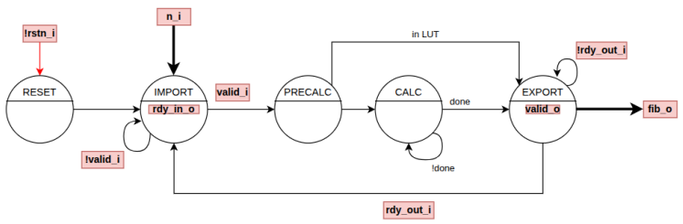


Figure 4.2: v1 Accelerator FSM

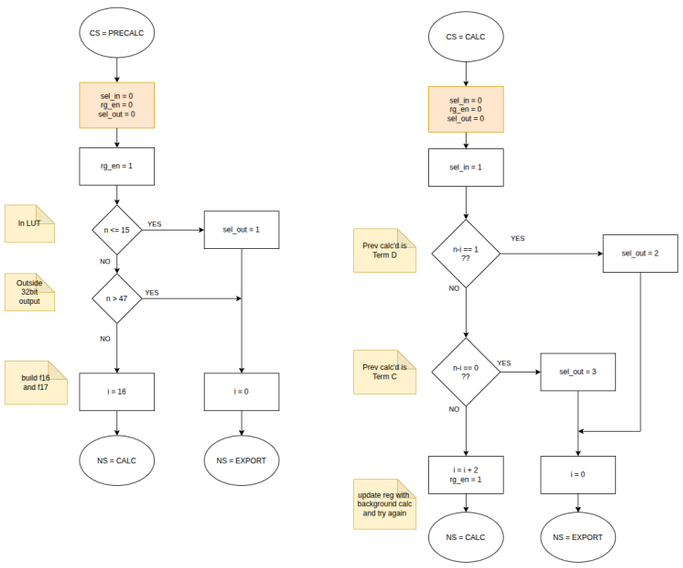


Figure 4.3: v1 Accelerator Flow

This redesign

- Removed an output 2-Mux and wired the registers directly to the output mux.
- Removed the `calc_done` logic and based the state logic around CU's internal signals.
- Broke up `CALC` logic into two states `{PRECALC, CALC}`, s.t. `PRECALC` handles the LUT values and pre-loads f_{14} and f_{15} into the registers, offsetting AND separating computation from outputting on the same cycle.
- Cleaned and simplified the internal logic in CU.

The logic is a bit cleaner:

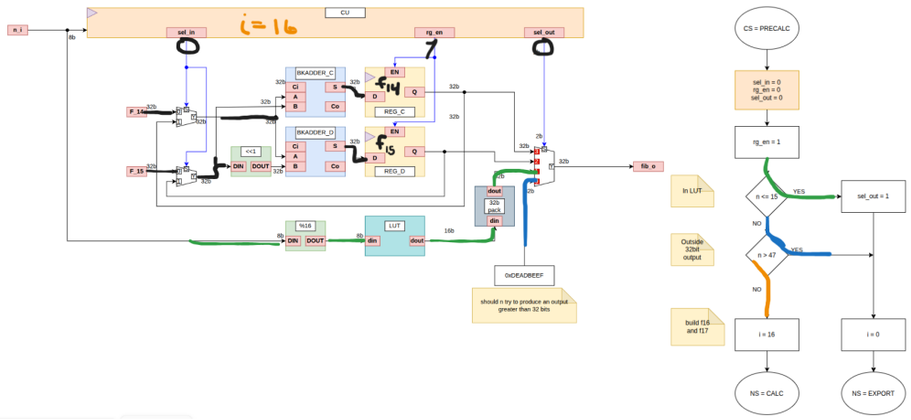


Figure 4.4: `PRECALC` flow

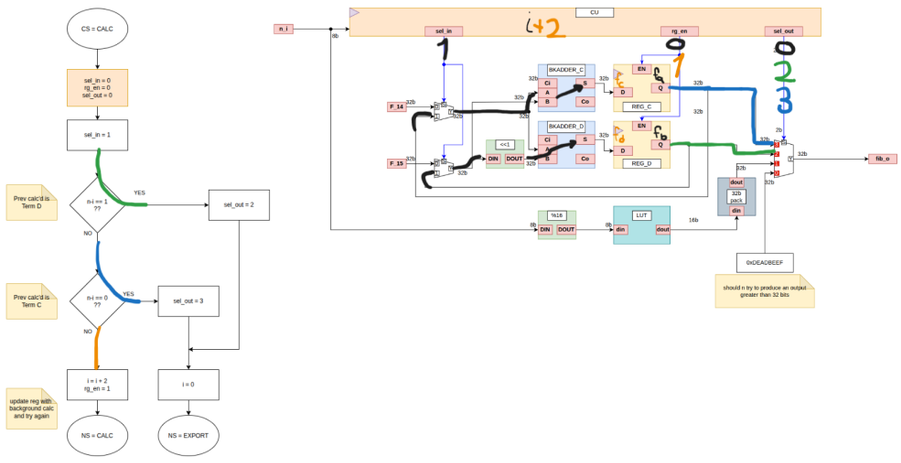


Figure 4.5: `CALC` flow

Note, the design meets all testing performed on v0.

4.1 v1 Post-Synthesis Analysis

Listing 4.1: v1 Pre-PNR Summary

```
+ analyze_power_grid -net VGND -voltage_file /home/whirck/Desktop/Sources/2026_Fall/class/CP470_OpenS_ASIC/02_labs/lab1-main/runs/RUN_2026-11-01_16-45-10/58-openroad-irdr
[INFO PSM-0040] All shapes on net VGND are connected.
########## IR report #################
Net              : VGND
Corner           : nom_tt_025C_1v80
Total power      : 3.75e-04 W
Supply voltage   : 0.00e+00 V
Worstcase voltage: 3.57e-04 V
Average voltage  : 8.43e-05 V
Average IR drop  : 8.43e-05 V
Worstcase IR drop: 3.57e-04 V
Percentage drop  : 0.02 %
######################################
Running 'Checker.SetupViolations' at 'runs/RUN_2026-11-01_16-45-10/74-checker-setupviolations'...
No setup violations found
Running 'Checker.HoldViolations' at 'runs/RUN_2026-11-01_16-45-10/75-checker-holdviolations'...
No hold violations found
Running 'Checker.MaxSlewViolations' at 'runs/RUN_2026-11-01_16-45-10/76-checker-maxslewviolations'...
Max Slew violations found in the following corners:
* max_ss_100C_1v60
* max_tt_025C_1v80
* min_ss_100C_1v60
* min_tt_025C_1v80
* nom_ss_100C_1v60
* nom_tt_025C_1v80
No max slew violations found
Running 'Checker.MaxCapViolations' at 'runs/RUN_2026-11-01_16-45-10/77-checker-maxcapviolations'...
No max cap violations found
```

Listing 4.2: v1 Pre-PNR STA Highlight

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violatio.	Max Slew Violati
Overall	0.1103	0.1103	0.0000	0	0	-0.5746	-0.5746	-1.9206	15	15	0	158
nom_tt_025C_1v80	0.3191	0.3191	0.0000	0	0	3.8645	4.6467	0.0000	0	0	0	11
nom_ss_100C_1v60	0.8997	0.8997	0.0000	0	0	-0.4540	-0.4540	-0.9837	4	4	0	136
nom_ff_n40C_1v95	0.1127	0.1127	0.0000	0	0	5.1901	6.5422	0.0000	0	0	0	0
min_tt_025C_1v80	0.3159	0.3159	0.0000	0	0	3.9926	4.7121	0.0000	0	0	0	0
min_ss_100C_1v60	0.8940	0.8940	0.0000	0	0	-0.3324	-0.3324	-0.6226	3	3	0	125
min_ff_n40C_1v95	0.1103	0.1103	0.0000	0	0	5.2728	6.5860	0.0000	0	0	0	0
max_tt_025C_1v80	0.3223	0.3223	0.0000	0	0	3.7504	4.5807	0.0000	0	0	0	22
max_ss_100C_1v60	0.9053	0.9053	0.0000	0	0	-0.5746	-0.5746	-1.9206	8	8	0	158
max_ff_n40C_1v95	0.1150	0.1150	0.0000	0	0	5.1120	6.4876	0.0000	0	0	0	0

Fig 4.1 states that we don't have setup or hold violations but fig 4.2 does still indicate a few hiccups, having negative slack in our slowest corner case. We've managed to reduce our biggest setup violation by more than a full clock cycle and fanout in fig 4.3 looks better as well, reducing 126 down to 16! Looking at our critical path, most of our delay is accumulated around driving the output signal `fib_o` as well as `mux_a`.

Listing 4.3: v1 Pre-PNR Biggest Setup Violator

```
====================== nom_ss_100C_1v60 Corner ===================================

Startpoint: CS[4]_sky130_fd_sc_hd__dfxtp_2_Q
            (rising edge-triggered flip-flop clocked by clk_i)
Endpoint: mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q
          (rising edge-triggered flip-flop clocked by clk_i)
Path Group: clk_i
Path Type: max

Fanout          Cap         Slew       Delay        Time   Description
---------------------------------------------------------------------------------------------
                                    0.000000    0.000000   clock clk_i (rise edge)
                                    0.000000    0.000000   clock source latency
     1    0.023071    0.171369    0.123488    0.123488 ^ clk_i (in)
                                                         clk_i (net)
                     0.171376    0.000000    0.123488 ^ clkbuf_0_clk_i/A (sky130_fd_sc_hd__clkbuf_16)
    16    0.103885    0.181448    0.382154    0.505642 ^ clkbuf_0_clk_i/X (sky130_fd_sc_hd__clkbuf_16)
                                                         clknet_0_clk_i (net)
                     0.181472    0.002455    0.508097 ^ clkbuf_4_12_0_clk_i/A (sky130_fd_sc_hd__clkbuf_8)
     6    0.020152    0.083848    0.301667    0.809764 ^ clkbuf_4_12_0_clk_i/X (sky130_fd_sc_hd__clkbuf_8)
                                                         clknet_4_12_0_clk_i (net)
                     0.083848    0.000315    0.810079 ^ CS[4]_sky130_fd_sc_hd__dfxtp_2_Q/CLK (sky130_fd_sc_hd__dfxtp_2)
     1    0.005806    0.072948    0.695636    1.505716 ^ CS[4]_sky130_fd_sc_hd__dfxtp_2_Q/Q (sky130_fd_sc_hd__dfxtp_2)
                                                         CS[4] (net)
                     0.072948    0.000239    1.505955 ^ fanout91/A (sky130_fd_sc_hd__dlymetal6s4s_1)
     7    0.044126    0.677551    0.776778    2.282733 ^ fanout91/X (sky130_fd_sc_hd__dlymetal6s4s_1)
                                                         net91 (net)
                     0.677561    0.002277    2.285010 ^ fanout81/A (sky130_fd_sc_hd__dlymetal6s4s_1)
     7    0.033591    0.525958    0.885021    3.170030 ^ fanout81/X (sky130_fd_sc_hd__dlymetal6s4s_1)
                                                         net81 (net)
                     0.525958    0.000606    3.170636 ^ fanout78/A (sky130_fd_sc_hd__clkdlybuf4s25_1)
     2    0.008507    0.157435    0.737560    3.908196 ^ fanout78/X (sky130_fd_sc_hd__clkdlybuf4s25_1)
                                                         net78 (net)
                     0.157435    0.000179    3.908375 ^ load_slew79/A (sky130_fd_sc_hd__clkbuf_4)
     9    0.042978    0.217692    0.416160    4.324534 ^ load_slew79/X (sky130_fd_sc_hd__clkbuf_4)
                                                         net79 (net)
                     0.217703    0.001904    4.326438 ^ fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_A2_sky130_fd_sc_hd
     2    0.008524    0.101074    0.188363    4.514801 v fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_A2_sky130_fd_sc_hd
                                                         fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_A2_sky130_fd_sc_hd
                     0.101074    0.000219    4.515020 v fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_A2_sky130_fd_sc_hd
     4    0.011757    0.196161    0.852083    5.367104 v fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_A2_sky130_fd_sc_hd
                                                         fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_A2_sky130_fd_sc_hd
                     0.196161    0.000276    5.367379 v fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_B1_sky130_fd_sc_hd
     2    0.010262    0.269721    1.513824    6.881203 v fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_B1_sky130_fd_sc_hd
                                                         fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_B1 (net)
                     0.269721    0.000389    6.881592 v fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_B1_sky130_fd_sc_hd
     2    0.016735    0.155120    0.707951    7.589543 v fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_B1_sky130_fd_sc_hd
                                                         fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_X_sky130_fd_sc_hd_
                     0.155132    0.001473    7.591017 v fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_X_sky130_fd_sc_hd_
     3    0.020657    0.201243    1.028458    8.619475 v fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_X_sky130_fd_sc_hd_
                                                         fib_o_sky130_fd_sc_hd__a21o_2_X_4_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o221a_2_A1_X_sky130_fd_sc_hd_
                     0.201243    0.001485    8.620960 v fib_o_sky130_fd_sc_hd__a21o_2_X_1_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o22a_2_A1_X_sky130_fd_sc_hd__
     3    0.012517    0.100925    0.476376    9.097337 v fib_o_sky130_fd_sc_hd__a21o_2_X_1_B1_sky130_fd_sc_hd__a21oi_2_Y_A1_sky130_fd_sc_hd__o22a_2_A1_X_sky130_fd_sc_hd__
                                                         mux_a.DIN[56]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a22o_2_X_B1 (net)
                     0.100925    0.000227    9.097564 v mux_a.DIN[58]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a22o_2_X_B1_sky130_fd_sc_hd__xnor2_2_Y_B_sky130_fd_sc
     2    0.012320    0.102820    0.476438    9.574001 v mux_a.DIN[58]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a22o_2_X_B1_sky130_fd_sc_hd__xnor2_2_Y_B_sky130_fd_sc
                                                         mux_a.DIN[58]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a22o_2_X_B1_sky130_fd_sc_hd__xnor2_2_Y_B (net)
                     0.102820    0.000188    9.574190 v mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_B_sky130_fd_s
     1    0.009146    0.088142    0.432300   10.006490 v mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_B_sky130_fd_s
                                                         mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_B (net)
                     0.088142    0.000315   10.006805 v mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_B (sky130_fd_
     1    0.002631    0.092203    0.238650   10.245456 v mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_Y (sky130_fd_
                                                         mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2 (net)
                     0.092203    0.000056   10.245511 v mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2 (sky130_fd_sc_hd__o211a_2)
     1    0.001882    0.064939    0.462589   10.708100 v mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_X (sky130_fd_sc_hd__o211a_2)
                                                         mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q_D (net)
                     0.064939    0.000054   10.708155 v mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q_D (sky130_fd_sc_hd__dfxtp_2)
                                            10.708155   data arrival time

                                    10.000000   10.000000   clock clk_i (rise edge)
                                     0.000000   10.000000   clock source latency
     1    0.023071    0.171369    0.111728   10.111728 ^ clk_i (in)
                                                         clk_i (net)
                     0.171376    0.000000   10.111728 ^ clkbuf_0_clk_i/A (sky130_fd_sc_hd__clkbuf_16)
    16    0.103885    0.181448    0.345758   10.457486 ^ clkbuf_0_clk_i/X (sky130_fd_sc_hd__clkbuf_16)
                                                         clknet_0_clk_i (net)
                     0.181924    0.006341   10.463826 ^ clkbuf_4_11_0_clk_i/A (sky130_fd_sc_hd__clkbuf_8)
     6    0.015547    0.073341    0.263147   10.726974 ^ clkbuf_4_11_0_clk_i/X (sky130_fd_sc_hd__clkbuf_8)
                                                         clknet_4_11_0_clk_i (net)
                     0.073344    0.000425   10.727398 ^ mux_a.DIN[59]_sky130_fd_sc_hd__dfxtp_2_Q/CLK (sky130_fd_sc_hd__dfxtp_2)
                                -0.250000   10.477398   clock uncertainty
                                 0.048156   10.525554   clock reconvergence pessimism
                                -0.271376   10.254177   library setup time
                                            10.254177   data required time
---------------------------------------------------------------------------------------------
                                            10.254177   data required time
                                           -10.708155   data arrival time
---------------------------------------------------------------------------------------------
                                            -0.453977   slack (VIOLATED)
```

From here, the easier solutions have been implemented, leaving only a few niche cuts and tunings available. Consider:

- **Manually reducing the time-padding constants** built into librelane/sky130, meeting STA in a naive fashion. This won't improve the logic, just lower the timing standards. Different chip makers might afford this to the design but it's unlikely (and ultimately not a good idea at this phase of the design).
- **Further analyze for improvement** will require more time, something is short supply for this project. Entertaining ideas s.a. adopting a unique configuration for the muxes, which appear to be a constant in our critical path, appear to have merit but are no longer free lunches. Further analysis will have to be performed.
- **Increasing the clock period** was ultimately the option implemented to meet timing - by increasing the period, we preserve our current logic and the sky130A standard by making our logic slightly slower.

4.2 v1 meeting STA

Listing 4.4: v1's New config file

```
---
DESIGN_NAME: fib
VERILOG_FILES: dir::rtl/*.sv
CLOCK_PERIOD: 15
CLOCK_PORT: clk_i
# Power Distribution Stuff
PDN_VOFFSET: 7
PDN_HOFFSET: 7
PDN_SKIPTRIM: true
RUN_POST_GRT_RESIZER_TIMING: true
RUN_POST_GRT_DESIGN_REPAIR: true

# Use our naming convention
SYNTH_AUTONAME: true
```

This change will slow the clock from 100MHz to 66MHz.

Listing 4.5: v1 Post-PNR Summary

```
+ analyze_power_grid -net VGND -voltage_file /home/whirck/Desktop/Sources/2026_Fall/class/CP470_OpenS_ASIC/02_labs/lab1-main/runs/RUN_2026-11-01_17-26-02/58-openroad-irdr
[INFO PSM-0040] All shapes on net VGND are connected.
########## IR report #################
Net             : VGND
Corner          : nom_tt_025C_1v80
Total power     : 5.46e-04 W
Supply voltage  : 0.00e+00 V
Worstcase voltage: 4.68e-04 V
Average voltage  : 1.25e-04 V
Average IR drop  : 1.25e-04 V
Worstcase IR drop: 4.68e-04 V
Percentage drop  : 0.03 %
######################################
Running 'Checker.SetupViolations' at 'runs/RUN_2026-11-01_17-26-02/74-checker-setupviolations'...
No setup violations found
Running 'Checker.HoldViolations' at 'runs/RUN_2026-11-01_17-26-02/75-checker-holdviolations'...
No hold violations found
Running 'Checker.MaxSlewViolations' at 'runs/RUN_2026-11-01_17-26-02/76-checker-maxslewviolations'...
Max Slew violations found in the following corners:
* max_ss_100C_1v60
* min_ss_100C_1v60
* nom_ss_100C_1v60
No max slew violations found
Running 'Checker.MaxCapViolations' at 'runs/RUN_2026-11-01_17-26-02/77-checker-maxcapviolations'...
No max cap violations found
```

Listing 4.6: v1 Post-PNR STA Highlight w/ Clock Change

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violatio.	Max Slew Violati..
Overall	0.1130	0.1130	0.0000	0	0	1.1227	1.1227	0.0000	0	0	0	126
nom_tt_025C_1v80	0.3391	0.3391	0.0000	0	0	8.2281	8.2864	0.0000	0	0	0	0
nom_ss_100C_1v60	0.9161	0.9161	0.0000	0	0	1.2442	1.2442	0.0000	0	0	0	95
nom_ff_n40C_1v95	0.1158	0.1158	0.0000	0	0	9.4239	N/A	0.0000	0	0	0	0
min_tt_025C_1v80	0.3352	0.3352	0.0000	0	0	8.2973	8.3544	0.0000	0	0	0	0
min_ss_100C_1v60	0.9079	0.9079	0.0000	0	0	1.3729	1.3729	0.0000	0	0	0	66
min_ff_n40C_1v95	0.1130	0.1130	0.0000	0	0	9.4710	N/A	0.0000	0	0	0	0
max_tt_025C_1v80	0.3433	0.3433	0.0000	0	0	8.1585	8.2177	0.0000	0	0	0	0
max_ss_100C_1v60	0.9237	0.9237	0.0000	0	0	1.1227	1.1227	0.0000	0	0	0	126
max_ff_n40C_1v95	0.1189	0.1189	0.0000	0	0	9.3759	N/A	0.0000	0	0	0	0

Listing 4.7: v1 Post-PNR Biggest Setup Violator w/ Clock Change

```
====================== max_ss_100C_1v60 Corner ===================================

Startpoint: CS[4]_sky130_fd_sc_hd__dfxtp_2_Q
            (rising edge-triggered flip-flop clocked by clk_i)
Endpoint: mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q
          (rising edge-triggered flip-flop clocked by clk_i)
Path Group: clk_i
Path Type: max

Fanout          Cap         Slew       Delay        Time   Description
---------------------------------------------------------------------------------------------------
                                    0.000000    0.000000   clock clk_i (rise edge)
                                    0.000000    0.000000   clock source latency
     1    0.025658    0.189506    0.137267    0.137267 ^ clk_i (in)
                                                         clk_i (net)
                     0.189522    0.000000    0.137267 ^ clkbuf_0_clk_i/A (sky130_fd_sc_hd__clkbuf_16)
    16    0.111053    0.192740    0.395938    0.533205 ^ clkbuf_0_clk_i/X (sky130_fd_sc_hd__clkbuf_16)
                                                         clknet_0_clk_i (net)
                     0.193957    0.012889    0.546094 ^ clkbuf_4_4_0_clk_i/A (sky130_fd_sc_hd__clkbuf_8)
     6    0.021135    0.086278    0.309192    0.855285 ^ clkbuf_4_4_0_clk_i/X (sky130_fd_sc_hd__clkbuf_8)
                                                         clknet_4_4_0_clk_i (net)
                     0.086350    0.002335    0.857620 ^ CS[4]_sky130_fd_sc_hd__dfxtp_2_Q/CLK (sky130_fd_sc_hd__dfxtp_2)
     1    0.002644    0.051734    0.671866    1.529486 ^ CS[4]_sky130_fd_sc_hd__dfxtp_2_Q/Q (sky130_fd_sc_hd__dfxtp_2)
                                                         CS[4] (net)
                     0.051734    0.000197    1.529684 ^ fanout94/A (sky130_fd_sc_hd__clkdlybuf4s25_1)
     5    0.038997    0.626652    0.897378    2.427061 ^ fanout94/X (sky130_fd_sc_hd__clkdlybuf4s25_1)
                                                         net94 (net)
                     0.626655    0.002737    2.429799 ^ fanout88/A (sky130_fd_sc_hd__clkdlybuf4s25_1)
    10    0.040374    0.649421    1.145360    3.575159 ^ fanout88/X (sky130_fd_sc_hd__clkdlybuf4s25_1)
                                                         net88 (net)
                     0.649421    0.001279    3.576438 ^ fanout87/A (sky130_fd_sc_hd__clkdlybuf4s25_1)
    10    0.041070    0.660679    1.158069    4.734507 ^ fanout87/X (sky130_fd_sc_hd__clkdlybuf4s25_1)
                                                         net87 (net)
                     0.660687    0.003470    4.737977 ^ mux_a.DIN[42]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__nor2_2_Y_A_sky130_fd_sc
     2    0.005947    0.161607    0.317475    5.055452 v mux_a.DIN[42]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__nor2_2_Y_A_sky130_fd_sc
                                                         mux_a.DIN[42]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__nor2_2_Y_A_sky130_fd_sc
                     0.161607    0.000291    5.055743 v mux_a.DIN[42]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__nor2_2_Y_A_sky130_fd_sc
     2    0.005823    0.245181    1.715385    6.771128 v mux_a.DIN[42]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__nor2_2_Y_A_sky130_fd_sc
                                                         mux_a.DIN[44]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_B_sky130_fd_sc
                     0.245181    0.000372    6.771500 v mux_a.DIN[44]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_B_sky130_fd_sc
     2    0.009605    0.203480    1.158329    7.929829 v mux_a.DIN[44]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_B_sky130_fd_sc
                                                         mux_a.DIN[44]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_B_sky130_fd_sc
                     0.203497    0.000881    7.930710 v mux_a.DIN[44]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_B_sky130_fd_sc
     1    0.007942    0.256842    1.758600    9.689309 v mux_a.DIN[44]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_B_sky130_fd_sc
                                                         mux_a.DIN[48]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_A_sky130_fd_sc
                     0.256842    0.000552    9.689860 v mux_a.DIN[48]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_A_sky130_fd_sc
     2    0.008849    0.084393    0.469071   10.158932 v mux_a.DIN[48]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_A_sky130_fd_sc
                                                         mux_a.DIN[48]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_A_sky130_fd_sc
                     0.084399    0.000787   10.159719 v mux_a.DIN[48]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_A_sky130_fd_sc
     3    0.020566    0.131499    0.462386   10.622105 v mux_a.DIN[48]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_A_sky130_fd_sc
                                                         mux_a.DIN[48]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xor2_2_X_A_sky130_fd_sc
                     0.131524    0.001833   10.623938 v mux_a.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__a21o_2_X_A1_sky130_fd_s
     2    0.008527    0.110797    0.656460   11.280397 v mux_a.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__a21o_2_X_A1_sky130_fd_s
                                                         mux_a.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__a21o_2_X_A1_sky130_fd_s
                     0.110797    0.000510   11.280908 v mux_a.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__a21o_2_X_A1_sky130_fd_s
     4    0.017392    0.124158    0.461085   11.741992 v mux_a.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__a21o_2_X_A1_sky130_fd_s
                                                         mux_a.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__a21o_2_X_A1_sky130_fd_s
                     0.124167    0.001174   11.743166 v mux_a.DIN[62]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__a21o_2_X_A1_sky130_fd_s
     2    0.008441    0.140154    0.767223   12.510388 v mux_a.DIN[62]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__a21o_2_X_A1_sky130_fd_s
                                                         mux_a.DIN[62]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__a21o_2_X_A1_sky130_fd_s
                     0.140154    0.000120   12.510509 v mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o311a_2_X_A2_sky130_fd_sc_hd__and3_2_X_B_sky130_fd_sc
     3    0.012854    0.102745    0.483331   12.993839 v mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o311a_2_X_A2_sky130_fd_sc_hd__and3_2_X_B_sky130_fd_sc
                                                         mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o311a_2_X_A2_sky130_fd_sc_hd__and3_2_X_B (net)
                     0.102748    0.000753   12.994593 v mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o311a_2_X_A2_sky130_fd_sc_hd__and3_2_X_B (sky130_fd_s
     1    0.004243    0.078554    0.447089   13.441682 v mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o311a_2_X_A2_sky130_fd_sc_hd__and3_2_X_X (sky130_fd_s
                                                         mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o311a_2_X_A2 (net)
                     0.078554    0.000314   13.441996 v mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o311a_2_X_A2 (sky130_fd_sc_hd__o311a_2)
     1    0.002850    0.092808    0.736367   14.178362 v mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o311a_2_X_X (sky130_fd_sc_hd__o311a_2)
                                                         mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q_D (net)
                     0.092808    0.000224   14.178586 v mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q_D (sky130_fd_sc_hd__dfxtp_2)
                                            14.178586   data arrival time

                                   15.000000   15.000000   clock clk_i (rise edge)
                                    0.000000   15.000000   clock source latency
     1    0.025658    0.189506    0.124194   15.124194 ^ clk_i (in)
                                                         clk_i (net)
                     0.189522    0.000000   15.124194 ^ clkbuf_0_clk_i/A (sky130_fd_sc_hd__clkbuf_16)
    16    0.111053    0.192740    0.358229   15.482423 ^ clkbuf_0_clk_i/X (sky130_fd_sc_hd__clkbuf_16)
                                                         clknet_0_clk_i (net)
                     0.193201    0.007294   15.489717 ^ clkbuf_4_9_0_clk_i/A (sky130_fd_sc_hd__clkbuf_8)
     6    0.024211    0.093691    0.285873   15.775589 ^ clkbuf_4_9_0_clk_i/X (sky130_fd_sc_hd__clkbuf_8)
                                                         clknet_4_9_0_clk_i (net)
                     0.093769    0.002311   15.777901 ^ mux_a.DIN[63]_sky130_fd_sc_hd__dfxtp_2_Q/CLK (sky130_fd_sc_hd__dfxtp_2)
                                   -0.250000   15.527902   clock uncertainty
                                    0.050781   15.578682   clock reconvergence pessimism
                                   -0.277426   15.301256   library setup time
                                               15.301256   data required time
---------------------------------------------------------------------------------------------------
                                               15.301256   data required time
                                              -14.178586   data arrival time
---------------------------------------------------------------------------------------------------
                                                1.122670   slack (MET)
```

We've arrived to a point where we don't have setup or hold violations nor do we have negative slack across the board, seen in fig 4.7. There are still slew timing violations but they are not as severe as the others. By slowing down the clock, we've chosen to make our accelerator more conservative and accept the critical path in its current state.

Chapter 5

Tile Area

The final part of this project was to see if we could fit it on a TinyTapeout tile of $160\mu m \times 100\mu m = 16000\mu m^2$. The calculated die area ended up being $20,762\mu m^2$, resulting in a $\sim 30\%$ overcast of TT's tile. Potential options for shrinking the accelerator could entail:

- Limiting the output range of our module to 16 or 8 bits.
- Storing less numbers in the LUT at the cost of more compute time.

Chapter 6

Appendix

6.1 Code

6.1.1 RTL

Listing 6.1: Mux

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 07/17/2026 04:24:06 PM
// Design Name:
// Module Name: Mux_N
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//                          din[0] occupies the LSB WIDTH slot
//////////////////////////////////////////////////////////////////////////////////


module Mux_N
#(
    parameter WIDTH     = 32,
    parameter INPUTS    = 4,
    parameter SEL_WIDTH = (INPUTS <= 1) ? 1 : $clog2(INPUTS)
)
(
    input logic     [INPUTS-1:0][WIDTH-1:0] DIN,
    input logic     [SEL_WIDTH-1:0]         SEL,
    output logic    [WIDTH-1:0]             DOUT
);

    always_comb begin

        // ~~ default ~~
        DOUT = '0;

        if(int'(SEL) < INPUTS) begin
            DOUT = DIN[SEL];
        end
    end

endmodule
```

Listing 6.2: Brent-Kung 'Parallel Prefix' Adder

```
/* verilator lint_off UNOPTFLAT */
`timescale 1ns / 1ps


module BKAdder
#( parameter int WIDTH = 32 )
(
    input  logic [WIDTH-1:0] a,
    input  logic [WIDTH-1:0] b,
    input  logic             cin,
    output logic [WIDTH-1:0] sum,
    output logic             cout
);

    // ~~~~ local vars ~~~~
    // Keep at least one tree level so WIDTH=1 is supported.
    localparam int LEVELS = (WIDTH <= 1) ? 1 : $clog2(WIDTH);
    localparam int N      = 1 << LEVELS;
    localparam int STAGES = 2*LEVELS - 1;

    wire [N-1:0] a_pad = {{(N-WIDTH){1'b0}}, a};
    wire [N-1:0] b_pad = {{(N-WIDTH){1'b0}}, b};

    // Prefix arrays: stage 0 contains each bit's individual terms.
    wire [N-1:0] P [0:STAGES];
    wire [N-1:0] G [0:STAGES];

```

```
    assign P[0] = a_pad ^ b_pad;
    assign G[0] = a_pad & b_pad;

    // ~~~~ Up-sweep: combine adjacent groups ~~~~
    for (genvar s = 0; s < LEVELS; s++) begin : gen_up
        localparam int SPAN = 1 << s;
        localparam int PREV = s;
        localparam int NEXT = s + 1;

        for (genvar i = 0; i < N; i++) begin : gen_bit
            if (((i + 1) % (2*SPAN)) == 0) begin : gen_combine
                assign G[NEXT][i] =
                    G[PREV][i] | (P[PREV][i] & G[PREV][i-SPAN]);
                assign P[NEXT][i] =
                    P[PREV][i] & P[PREV][i-SPAN];
            end else begin : gen_pass
                assign G[NEXT][i] = G[PREV][i];
                assign P[NEXT][i] = P[PREV][i];
            end
        end
    end

    // ~~~~ Down-sweep: fill in the remaining prefixes ~~~~
    for (genvar t = 0; t < LEVELS-1; t++) begin : gen_down
        localparam int D    = LEVELS - 2 - t;
        localparam int SPAN = 1 << D;
        localparam int PREV = LEVELS + t;
        localparam int NEXT = PREV + 1;

        for (genvar i = 0; i < N; i++) begin : gen_bit
            if ((i >= 3*SPAN - 1) &&
                (((i - (3*SPAN - 1)) % (2*SPAN)) == 0)) begin : gen_combine
                assign G[NEXT][i] =
                    G[PREV][i] | (P[PREV][i] & G[PREV][i-SPAN]);
                assign P[NEXT][i] =
                    P[PREV][i] & P[PREV][i-SPAN];
            end else begin : gen_pass
                assign G[NEXT][i] = G[PREV][i];
                assign P[NEXT][i] = P[PREV][i];
            end
        end
    end

    wire [N-1:0] carry;

    // ~~~~ collect sum and carry out bit ~~~~
    for (genvar k = 0; k < WIDTH; k++) begin : gen_sum
        if (k == 0) begin : gen_first
            assign carry[k] = cin;
        end else begin : gen_other
            assign carry[k] =
                G[STAGES][k-1] | (P[STAGES][k-1] & cin);
        end

        assign sum[k] = P[0][k] ^ carry[k];
    end

    assign cout =
        G[STAGES][WIDTH-1] | (P[STAGES][WIDTH-1] & cin);


    `ifdef FORMAL
        always @(*) begin
            assert( (a + b + cin) == {cout, sum});
        end
    `endif

endmodule
```

Listing 6.3: Fibonacci Accelerator

```
`timescale 1ns / 1ps

/*
* Purpose : creating a fibonacci seq
*
* Inputs :
*       clk_i       : clock w/ pos active edge
*       rstn_i      : active low synch reset (resets on zero)
*       n_i         : nth fib sequence to stop at
*       valid_i     : driven high when n_i is ready to be read
*       rdy_out_i   : driven high when module's output has been read
*
* Returns :
*       rdy_in_o    : driven high by module to request new input data
*       fib_o       : calculated Fibonacci[n]
*       valid_o     : driven high by module when output data is ready
*/

module fib
#(
    parameter INBITS    = 8,
    parameter OUTBITS   = 32
)
(
    input logic                 clk_i,
    input logic                 rstn_i,
    input logic [INBITS-1:0]    n_i,
    input logic                 valid_i,
    input logic                 rdy_out_i,

    output logic                rdy_in_o,
    output logic [OUTBITS-1:0]  fib_o,
    output logic                valid_o
);


```

```
    // ~~~~ local vars ~~~~
    typedef enum logic [2:0] {RESET, IMPORT, PRECALC, CALC, EXPORT} e_state;
    e_state CS, NS /*verilator fsm_state*/;

    localparam logic [7:0] LUTMAX       = 8'h0F; //15
    localparam logic [7:0] NMAX         = 8'h2F; //47
    localparam logic [31:0] INACTIVE    = 32'hFFFF_FFFF;

    function logic [15:0] LUT (input logic [3:0] idx);
        case(idx)
            4'h0:       LUT = 16'h00_00;
            4'h1:       LUT = 16'h00_01;
            4'h2:       LUT = 16'h00_01;
            4'h3:       LUT = 16'h00_02;
            4'h4:       LUT = 16'h00_03;
            4'h5:       LUT = 16'h00_05;
            4'h6:       LUT = 16'h00_08;
            4'h7:       LUT = 16'h00_0D; //13
            4'h8:       LUT = 16'h00_15; //21
            4'h9:       LUT = 16'h00_22; //34
            4'hA:       LUT = 16'h00_37; //55
            4'hB:       LUT = 16'h00_59; //89
            4'hC:       LUT = 16'h00_90; //144
            4'hD:       LUT = 16'h00_E9; //233
            4'hE:       LUT = 16'h01_79; //377
            4'hF:       LUT = 16'h02_62; //610
            default:    LUT = 16'hDEAD;
        endcase
    endfunction

    logic               sel_in,
                        rg_en;
    logic [1:0]         sel_out_next,
                        sel_out_curr;
    logic [OUTBITS-1:0] regC, regD;
    logic [INBITS-1:0]  n_curr,
                        n_next,
                        i,
                        i_next;


    // ~~~~ module instantiations ~~~~

    logic [1:0][OUTBITS-1:0]mux_din_a,
                            mux_din_b;
    logic [3:0][OUTBITS-1:0]mux_din_c;
    logic [OUTBITS-1:0]     mux_dout_a,
                            mux_dout_b;
    wire [OUTBITS/2 -1:0]   lut_val;


    assign mux_din_a[0] = {16'h0, 16'h01_79};
    assign mux_din_a[1] = regC;


    assign mux_din_b[0] = {16'h0, 16'h02_62};
    assign mux_din_b[1] = regD;

    assign lut_val      = LUT(n_curr[3:0]);

    assign mux_din_c[0] = INACTIVE;
    assign mux_din_c[1] = {16'h0, lut_val};
    assign mux_din_c[2] = regD;
    assign mux_din_c[3] = regC;

    Mux_N
    #(
        .WIDTH      (OUTBITS),
        .INPUTS     (2),
        .SEL_WIDTH  ()
    )
    mux_a
    (
        .DIN        (mux_din_a),
        .SEL        (sel_in),
        .DOUT       (mux_dout_a)
    );

    Mux_N
    #(
        .WIDTH      (OUTBITS),
        .INPUTS     (2),
        .SEL_WIDTH  ()
    )
    mux_b
    (
        .DIN        (mux_din_b),
        .SEL        (sel_in),
        .DOUT       (mux_dout_b)
    );

    Mux_N
    #(
        .WIDTH      (OUTBITS),
        .INPUTS     (4),
        .SEL_WIDTH  ()
    )
    mux_fib
    (
        .DIN        (mux_din_c),
        .SEL        (sel_out_curr),
        .DOUT       (fib_o)
    );

    logic [OUTBITS-1:0] c_sum, d_sum;
    logic               unused1, unused2;

    BKAdder
```

```
    #( .WIDTH(OUTBITS))
    AddrC
    (
        .a(mux_dout_a),
        .b(mux_dout_b),
        .cin(1'b0),
        .sum(c_sum),
        .cout(unused1)
    );

    BKAdder
    #( .WIDTH(OUTBITS))
    AddrD
    (
        .a(mux_dout_a),
        .b(mux_dout_b << 1),
        .cin(1'b0),
        .sum(d_sum),
        .cout(unused2)
    );


    // ~~~~ synch logic ~~~~
    always_ff @(posedge clk_i) begin

        // ~~ priority 0 : reset
        if(rstn_i == 1'b0 ) begin
            i               <= '0;
            n_curr          <= '0;
            sel_out_curr    <= '0;
            regC            <= '0;
            regD            <= '0;
            CS              <= RESET;
        end

        // ~~ priority 1 : state wiggles ~~
        else begin
            i               <= i_next;
            n_curr          <= n_next;
            sel_out_curr    <= sel_out_next;

            if(rg_en) begin
                regC        <= c_sum;
                regD        <= d_sum;
            end

            CS              <= NS;
        end
    end



    // ~~~~ comb logic ~~~~
    always_comb begin

        // ~~ system defaults ~~
        rdy_in_o        = 1'b0;
        valid_o         = 1'b0;
        i_next          = '0;
        n_next          = n_curr;

        // ~ CALC defaults ~
        sel_in          = 1'b0;
        rg_en           = 1'b0;
        sel_out_next    = sel_out_curr;

        case(CS)

            RESET:
            begin
                sel_out_next    = '0;
                n_next          = '0;
                NS              = IMPORT;
            end

            IMPORT:
            begin
                sel_out_next    = '0;
                rdy_in_o        = 1'b1;

                if(valid_i) begin
                    n_next      = n_i;
                    NS          = PRECALC;
                end
                else begin
                    n_next      = '0;
                    NS          = IMPORT;
                end
            end

            PRECALC:
            begin

                rg_en               = 1'b1;     // store f16 and f17

                // Priority 0: n is inside LUT
                if(n_curr <= LUTMAX) begin
                    sel_out_next    = 2'b01;
                    NS              = EXPORT;
                end

                // Priority 1: (edge case) fib will be greater than OUTBITS
                else if(n_curr > NMAX) begin
                    sel_out_next    = 2'b0;
                    NS              = EXPORT;
                end
                // Priority 2: n is between [16,47]
                else begin
```

```
                    sel_out_next    = 2'b0;     // default to inactive
                    i_next          = 8'h10;    // init counting term
                    NS              = CALC;
                end

            end

            CALC:
            begin

                sel_in  = 1'b1; // use prev calc

                // Priority 0: found term C
                if( (n_curr - i) == 0 ) begin
                    sel_out_next    = 2'b11;
                    NS              = EXPORT;
                end
                // Priority 1: found term D
                else if((n_curr - i) == 1 ) begin
                    sel_out_next    = 2'b10;
                    NS              = EXPORT;
                end
                // Priority 2: keep counting
                else begin
                    rg_en           = 1'b1;     // capture current adder calc
                    i_next          = i + 8'h02;
                    NS              = CALC;
                end

            end

            EXPORT:
            begin
                valid_o                     = 1'b1;
                if(rdy_out_i == 1'b1)   NS  = IMPORT;
                else                    NS  = EXPORT;
            end

            default:
            begin
                NS          = RESET;
            end
        endcase
    end

    `ifdef FORMAL

        // =======================
        // locals
        // =======================
        localparam logic [31:0] FIBMAX      = 32'd2971215073;
        logic                   f_past_valid;
        initial                 f_past_valid = 0;
        always @(posedge clk_i) f_past_valid <= 1;
        always_comb             if(!f_past_valid) assume (!rstn_i);

        logic   f_waiting,
                f_reading,
                f_calcing,
                f_writing;

        assign  f_waiting = rdy_in_o && !valid_i;
        assign  f_reading = rdy_in_o && valid_i;
        assign  f_calcing = !rdy_in_o && !valid_o;
        assign  f_writing = !rdy_in_o && valid_o;

        // =======================
        // Fib Boundary Checks (TODO not passing Ind)
        // =======================

        // A natural zero output means the input must also be zero
        always @(posedge clk_i) begin
            if( (fib_o == 32'h0 && $rose(f_writing)) &&
                (rstn_i && $stable(rstn_i)) &&
                (f_past_valid && $stable(f_past_valid))
            ) begin
                assert($past(n_i, 2) == 8'h0);
                assert($past(f_reading, 2));
            end
        end

        // A natural invalid output means the input went over the max
        always @(posedge clk_i) begin
            if(f_past_valid && (fib_o == INACTIVE && $rose(f_writing))) begin
                assert($past(n_i,2) > NMAX);
                assert($past(f_reading, 2));
            end
        end

        // =======================
        // Data Integrity (TODO not passing Ind)
        // =======================
        function automatic longint unsigned fibboni(
            input logic [INBITS-1:0] n
        );
            longint unsigned a, b, t;
            a       = 0;
            b       = 1;
            for(int i = 0; i < NMAX; i++) begin
                if(i < n) begin
                    t = a + b;
                    a = b;
                    b = t;
                end
            end
            fibboni = a;
        endfunction

```

```
        // Ensure data is coming from equation
        (* keep *) logic [OUTBITS-1:0]  fib_expected;
        always @(posedge clk_i) begin
            if(
                f_past_valid        &&
                $rose(f_writing)    &&
                $stable(rstn_i)     &&
                fib_o != INACTIVE
            ) begin

                if      ($past(f_reading, 1)) begin
                    fib_expected = 32'(fibboni($past(n_i, 1)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 2)) begin
                    fib_expected = 32'(fibboni($past(n_i, 2)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 3)) begin
                    fib_expected = 32'(fibboni($past(n_i, 3)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 4)) begin
                    fib_expected = 32'(fibboni($past(n_i, 4)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 5)) begin
                    fib_expected = 32'(fibboni($past(n_i, 5)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 6)) begin
                    fib_expected = 32'(fibboni($past(n_i, 6)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 7)) begin
                    fib_expected = 32'(fibboni($past(n_i, 7)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 8)) begin
                    fib_expected = 32'(fibboni($past(n_i, 8)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 9)) begin
                    fib_expected = 32'(fibboni($past(n_i, 9)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 10)) begin
                    fib_expected = 32'(fibboni($past(n_i, 10)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 11)) begin
                    fib_expected = 32'(fibboni($past(n_i, 11)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 12)) begin
                    fib_expected = 32'(fibboni($past(n_i, 12)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 13)) begin
                    fib_expected = 32'(fibboni($past(n_i, 13)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 14)) begin
                    fib_expected = 32'(fibboni($past(n_i, 14)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 15)) begin
                    fib_expected = 32'(fibboni($past(n_i, 15)));
                    assert(fib_o == fib_expected);
                end

                else if ($past(f_reading, 16)) begin
                    fib_expected = 32'(fibboni($past(n_i, 16)));
                    assert(fib_o == fib_expected);
                end

                // should have seen a past read
                else begin
                    assert(0);
                end


            end
        end

        // ======================
        // Liveness Checking
        // ======================
        always@(posedge clk_i) if (rstn_i) begin
            if($past(rstn_i)) begin
                if      ($past(f_waiting) && valid_i)
                   assert(f_reading);
                else if ($past(f_reading))
                    assert(f_calcing);
```

```
                else if ($past(f_calcing) && valid_o)
                    assert(f_writing);
                else if ($past(f_writing) && valid_i && rdy_in_o)
                    assert(f_reading);
                else if ($past(f_writing) && !valid_i && rdy_in_o)
                    assert(f_waiting);

            end
        end

        // ========================
        // Induction Hardening
        // ========================
/*
        always@(*) if (!f_past_valid) begin
            assume(lut_val == LUT(n_curr[3:0]));
        end

        // ensure LUT is assigned accurately
        always @(posedge clk_i) if (f_past_valid) begin
            assert(mux_din_c[1] == {16'h0, lut_val});
        end
*/


    `endif

endmodule
```

6.1.2 Simulation SVA Testing

Listing 6.4: Testbench simulation for Fibonacci Accelerator

```
`timescale 1ns / 1ps

module tb_fib();

    // ~~~~ testing ~~~~
    `ifdef LOCALTEST
        import tb_utils_pkg::*;
        testcase tc;
        logic [31:0] tnum;
    `endif

    // ~~~~ locals ~~~~
    localparam      CLK_PERIOD = 15;
    logic           clk,
                    resetn,
                    n_is_valid,
                    fib_was_read,
                    rdy_new_n,
                    fib_is_valid;

    logic [7:0]     n;
    logic [31:0]    fib_actual, fib_expected;
    localparam  logic [31:0]ERR_CODE    = 32'hFFFF_FFFF;
    localparam  logic [7:0] INBITS      = 8;
    localparam  logic [7:0] NMAX        = 8'd47;

    // ~~~~ Gate-level ~~~~
    `ifdef USE_POWER_PINS
        wire VPWR;
        wire VGND;
        assign VPWR = 1;
        assign VGND = 0;
    `endif

    // ~~~~ functions ~~~~
    function automatic longint unsigned fibboni(
        input logic [INBITS-1:0] N
    );
        longint unsigned a, b, t;
        a       = 0;
        b       = 1;
        for(int unsigned i = 0; 8'(i) < NMAX; i++) begin
            if(i < N) begin
                t = a + b;
                a = b;
                b = t;
            end
        end
        fibboni = a;
    endfunction

    // ~~~~ tasks ~~~~
    task automatic request_fn(
        input logic [7:0]   n_val
    );
        // ~ IMPORT ~
        do begin
            @(posedge clk);
            fib_was_read = 1'b1;
        end
        while(rdy_new_n !== 1'b1);
        fib_was_read = 1'b0;

        n               = n_val;
        n_is_valid      = 1'b1;
        @(posedge clk);

        // ~ CALC -> EXPORT ~
        do @(posedge clk);
        while(fib_is_valid !== 1'b1);
        n_is_valid      = 1'b0;
        @(posedge clk);
    endtask

```

```
    // ~~~~ instantiations ~~~~
    fib
    `ifndef GATELEVEL
    #(
        .INBITS     (8),
        .OUTBITS    (32)
    )
    `endif
    Fib
    (
        .clk_i      (clk),
        .rstn_i     (resetn),
        .n_i        (n),
        .valid_i    (n_is_valid),
        .rdy_out_i  (fib_was_read),

        .rdy_in_o   (rdy_new_n),
        .fib_o      (fib_actual),
        .valid_o    (fib_is_valid)

        `ifdef GATELEVEL
        ,
        .VPWR(VPWR),
        .VGND(VGND)
        `endif
    );

    // ~~~~ heartbeat ~~~~
    always #(CLK_PERIOD/2) clk = ~clk;

    `ifdef USE_SVA
        // ~~ sequences (unsupported by icarus) ~~

        // ~~ properties (unsupported by icarus) ~~
        property p1;
            @(posedge clk)
                $rose(resetn) |-> $rose(rdy_new_n);
        endproperty

        property p2;
            @(posedge clk) disable iff(!resetn)
                not(rdy_new_n && fib_is_valid);
        endproperty

        property p3;
            @(posedge clk) disable iff(!resetn)
                ($past(fib_is_valid) && fib_is_valid && !fib_was_read) |->
                    ($past(fib_actual) == fib_actual);
        endproperty

        // ~~ concurrents (unsupported by icarus)~~
        reset_import    : assert property(p1) else begin
            `ifdef  LOCALTEST tc.err("Didn't enter IMPORT after RESET");
            `else   $display("Didn't enter IMPORT after RESET");
            `endif
        end

        import_export   : assert property(p2) else begin
            `ifdef  LOCALTEST tc.err("Shouldn't accept new data while holding old");
            `else   $display("Shouldn't accept new data while holding old");
            `endif
        end

        hold_fib        : assert property(p3) else begin
            `ifdef  LOCALTEST tc.err("fib output wasn't held");
            `else   $display("fib output wasn't held");
            `endif
        end
    `endif

    // ~~~~ time-dependent testing ~~~~
    initial begin

        // ~~ waveforms ~~
        $dumpfile("tb_fib.vcd");
        $dumpvars(0);

        // ~~ defaults ~~

        clk             = 1'b1;
        resetn          = 1'b0;
        n               = 8'h00;
        n_is_valid      = 1'b0;
        fib_was_read    = 1'b0;

        `ifdef LOCALTEST tc = new(); `endif

        // ~~ fresh start ~~
        repeat(10) begin @(posedge clk); end
        resetn          = 1'b1;

        // ~~ immediate ~~
        `ifdef LOCALTEST
        tc.new_test("n in [0,47]");
        tnum = tc.get_testnum();
        `endif

            for(logic [7:0] i = 8'h00; i < 8'd48; i = i + 8'h01) begin

                request_fn(.n_val(i));
                fib_expected = 32'(fibboni(n));
                assert(fib_expected === fib_actual) else begin
                    `ifdef  LOCALTEST tc.err("fib value didn't match");
                    `else   $display("fib value didn't match");
                    `endif
                end

                repeat(2) begin @(posedge clk); end
```

```
                fib_was_read    = 1'b1;
                @(posedge clk);
                fib_was_read    = 1'b0;
            end
            @(posedge clk);
        `ifdef LOCALTEST tc.test_done(); `endif

        `ifdef LOCALTEST
        tc.new_test("n in 48+");
        tnum = tc.get_testnum();
        `endif
            for(logic [7:0] i = 8'd48; i < 8'hFF; i = i + 8'h01) begin

                request_fn(.n_val(i));
                assert(fib_actual === ERR_CODE) else begin
                    `ifdef  LOCALTEST tc.err(" ");
                    `else   $display("fib value didnt match");
                    `endif
                end

                repeat(2) begin @(posedge clk); end
                fib_was_read    = 1'b1;
                @(posedge clk);
                fib_was_read    = 1'b0;
                @(posedge clk);
            end
        `ifdef LOCALTEST tc.test_done(); `endif

        $finish();
    end
endmodule
```

Chapter 7

References

1. CPE490 - OpenSource ASIC